



High Performance Computing 2026S Assignment 2

June 9, 2026

3 Person Group 12

1: Aman BHARDWAJ, 12333472
2: Muskan SODHI, 12436439
3: Sanket UNDRE, 12347493

1 Exercise1: Implementing CUDA Streams

1.1 Task 1: Implementation of Version A and Version B

In this task, we implemented two distinct approaches to process a large array of floats on the GPU, focusing on memory management and execution concurrency.

Version A: Synchronous: (No Streams) In our implementation of `run_gpu_nostream`, we followed a standard synchronous processing pattern. We allocated device memory for the entire array size using `cudaMalloc`, performed a bulk `cudaMemcpy` from host to device, and then launched a single CUDA kernel grid to execute the `transform_kernel` over the entire input. After the kernel completed, we performed a synchronous `cudaMemcpy` from device back to host. This approach, while simple, serves as our baseline; it enforces a strictly sequential execution where the GPU remains idle during the large data transfer phases, and no concurrency between memory movement and computation is exploited.

Version B: Asynchronous (Stream-Based) To optimize performance, we implemented `run_gpu_streams` using a pipelined approach to overlap memory transfers with kernel computation:

- **Data Chunking:** We partitioned the total input array into $N_{CHUNKS} = 16$ equal segments. This allows us to process the data in smaller increments, which is necessary for pipelining.
- **Stream Pipelining:** We utilized a pool of CUDA streams (up to 8) to manage the execution order. For each chunk, we issued a `cudaMemcpyAsync` for the Host-to-Device transfer, followed by an asynchronous kernel launch, and finally a `cudaMemcpyAsync` for the Device-to-Host transfer, all assigned to the same stream.

- **Overlap Mechanism:** By cycling through the available streams using the modulo operator ($i \% n_streams$), we effectively hide the latency of memory transfers behind the execution of kernels in other streams. The GPU can perform the transformation on one chunk while simultaneously transferring the input/output data for another chunk.

Finally, for both versions, we ensured correctness by verifying that our GPU results were consistent with the CPU-side prefix sum implementation. We used `cudaDeviceSynchronize()` in Version B to ensure all asynchronous tasks were completed before performing the final host-side prefix sum calculation, ensuring our implementation passed the `results_match()` verification.

In both implementations, we ensured kernel efficiency by utilizing a grid size calculated based on the `BLOCK_SIZE` and array size, ensuring full occupancy and coalesced memory access patterns within the `transform_kernel`.

1.2 Task 2: Results from Running the Codes and performance Analysis

To evaluate the performance and scalability of our CUDA stream implementation, we benchmarked the pipeline across three distinct computational workloads configured by `TRANSFORM_ITERS` values of 1, 50, and 200. To ensure statistical reliability and mitigate environmental variance on the shared JupyterHub infrastructure, we created an automated bash script, `run_benchmarks.sh`. This script executed our compiled binary five consecutive times for each configuration, captured the raw output metrics, and calculated the arithmetic mean execution times and corresponding speedup factors relative to the single-threaded CPU baseline. The structural data collected from these automated profiling iterations are detailed in Tables 1, 2, and 3.

Table 1: Performance and Speedup Metrics for `TRANSFORM_ITERS = 1`

Configuration	Mean Time (ms)	Speedup vs. CPU
CPU reference	924.65	–
Version A, no streams	950.60	0.97x
Version B, 1 stream	379.22	2.44x
Version B, 2 streams	312.48	2.96x
Version B, 4 streams	313.87	2.95x
Version B, 8 streams	310.51	2.98x

As summarized in Table 1, when the transformation workload is set to 1 iteration, the total computational intensity per data element is exceptionally light. Under this configuration, the execution profile is entirely dominated by data movement latencies. We observe that Version A yields a negative speedup (0.97x), failing to outperform the CPU due to the steep, non-overlapped overhead of synchronous host-to-device and device-to-host memory allocations and transfers. Conversely, Version B significantly mitigates this memory bottleneck even at a low stream count, achieving a 2.44x speedup with 1 stream and topping out at 2.98x speedup with 8 streams. This demonstrates that chunking data allows the hardware to pipeline operational phases effectively when compute requirements are minimal.

Table 2: Performance and Speedup Metrics for TRANSFORM_ITERS = 50

Configuration	Mean Time (ms)	Speedup vs. CPU
CPU reference	14592.50	–
Version A, no streams	1060.85	13.76x
Version B, 1 stream	388.21	37.59x
Version B, 2 streams	319.77	45.63x
Version B, 4 streams	318.87	45.76x
Version B, 8 streams	312.00	46.77x

Table 2 tracks performance when the workload is scaled up to 50 iterations. Here, the sequential execution runtime of the CPU reference spikes dramatically to over 14.5 seconds (14592.50 ms), shifting the application profile toward a compute-heavy envelope. With the GPU threads heavily saturated by mathematical tasks, Version A leverages massive hardware parallelism to achieve a notable 13.76x speedup over the CPU baseline. However, the architectural decoupling in Version B widens the performance gap further, delivering a 37.59x speedup with a single stream and a commanding 46.77x speedup utilizing 8 streams. This trend highlights that hiding memory transit latency becomes increasingly impactful as execution kernels expand in duration.

Table 3: Performance and Speedup Metrics for TRANSFORM_ITERS = 200

Configuration	Mean Time (ms)	Speedup vs. CPU
CPU reference	56746.15	–
Version A, no streams	980.35	57.88x
Version B, 1 stream	414.61	136.87x
Version B, 2 streams	317.94	178.48x
Version B, 4 streams	312.47	181.61x
Version B, 8 streams	309.62	183.28x

At 200 iterations, as observed from Table 3, the application transitions completely into a compute-bound state. While Version A scales predictably to a 57.88x speedup, Version B yields a profound performance leap, capturing a 136.87x speedup with 1 stream and peaking at an outstanding 183.28x speedup across 8 streams. At this high computational depth, the duration of the execution kernels heavily outweighs the data transit windows. This structural equilibrium allows the stream pipeline to run at peak saturation, hiding virtually all memory transfer latencies behind active thread blocks.

Figure 1 illustrates the execution runtime behavior of Version B across the three configurations. A significant performance improvement is observed when scaling from 1 stream to 2 streams, where execution times drop sharply from roughly 380–415 ms down to approximately 312–320 ms. This drop highlights the effective overlapping of Host-to-Device/Device-to-Host memory operations with kernel computation. Beyond 2 streams, the performance lines plateau, showing minor fluctuations but overall converging near 309 ms at 8 streams.

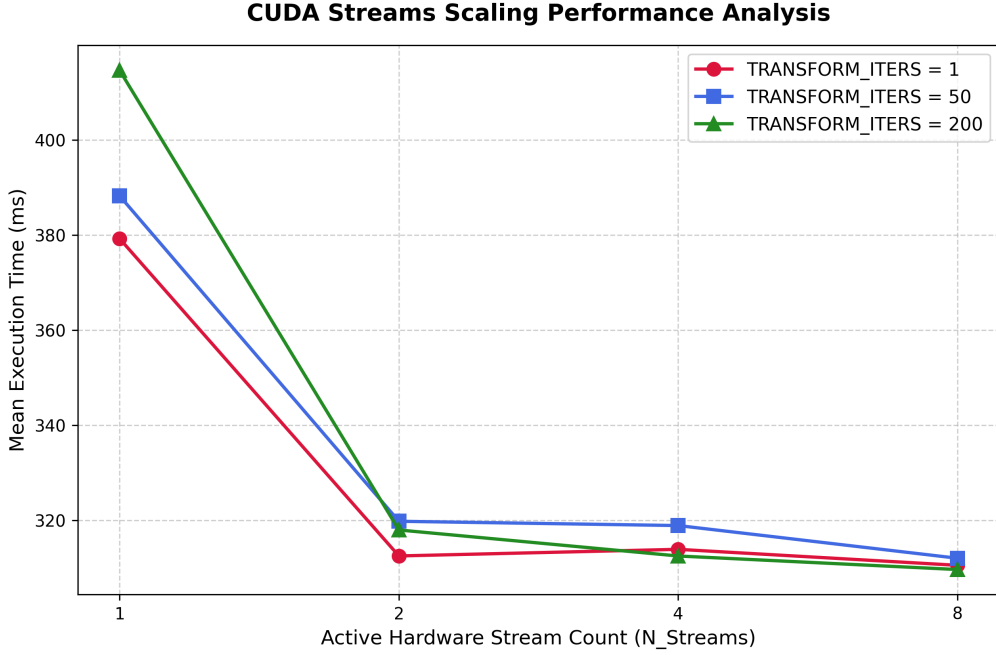


Figure 1: Mean execution time (ms) as a function of active hardware CUDA streams across varying computational intensities for Version B ($TRANSFORM_ITERS = 1, 50, 200$).

1.2.1 Answers to Questions

Answer 1: Optimal Stream Count and Stability Across Workloads

Our empirical measurements demonstrate that a configuration of 8 streams consistently yields the lowest execution runtimes and peak speedup factors across all evaluated workloads with ($TRANSFORM_ITERS = 1, 50, 200$). The optimal stream count remains remarkably stable and does not change despite the scaling of computational depth.

This stability is fundamentally governed by the hardware capabilities of the Tesla T4 GPU and the mechanics of the CUDA driver’s hardware work queues. By partitioning our global dataset into $N_{CHUNKS} = 16$ blocks, distributing these chunks across an increasing number of streams allows us to achieve deep pipeline concurrency. When scaling from 1 to 2, and up to 8 streams, we maximize the hardware’s capacity to concurrently exploit its asynchronous Dual Copy Engines (handling simultaneous Host-to-Device and Device-to-Host transfers via the PCIe bus) alongside the Execution Engine (the Streaming Multiprocessors).

The reason the optimal point does not change when shifting from a memory-bound scenario ($ITERS = 1$) to a heavily compute-bound scenario ($ITERS = 200$) comes down to latency hiding thresholds:

- At **low iterations**, the execution timeline is highly sparse; we need a high stream count (8) to ensure that as soon as one chunk finishes transferring, the next is immediately ready, keeping the copy engines fully saturated.
- At **high iterations**, the kernel execution duration is long enough to easily hide the

memory transfer of subsequent chunks. However, maintaining 8 streams populates the GPU’s hardware queues with multiple pending commands. This deep queue allows the internal hardware scheduler to hide instruction-level latencies and context-switch overhead seamlessly.

Once we hit 8 streams, the physical capability of the Tesla T4 to mask operational overhead reaches strict saturation. Any further increase in the stream pool would yield diminishing returns, as the underlying hardware resources (PCIe lanes and execution pipelines) are already operating at maximum utilization, keeping the optimal configuration stable across all workloads.

Answer 2: Performance Evaluation of Version B (1 Stream) vs. Version A (No Streams)

Our benchmarking data reveals a sharp performance divergence between Version B running on 1 stream and the synchronous Version A. Across all three workloads, Version B (1 stream) dramatically outperforms Version A, cutting execution runtimes by more than half in memory-dominated configurations and more than tripling execution performance in compute-bound scenarios. This behavior is initially counterintuitive since both configurations execute code sequentially without relying on multi-stream concurrency.

We attribute this significant performance delta to two primary architectural drivers:

- **Asynchronous Pipelining and Page-Locked Host Allocation:** Version B utilizes pinned host memory via `cudaMallocHost`, which permits vastly higher data transfer bandwidth through the PCIe bus compared to the standard pageable host allocations deployed in Version A.
- **Driver Queue Efficiencies and Cache Locality:** By breaking data into smaller, localized chunks, Version B avoids huge, blocking driver commands. The hardware processing engines interact with localized cache pools more efficiently. This setup allows the execution of individual chunk kernels to start up rapidly in parallel with minimal driver-side orchestration overhead, avoiding the systemic, blocking delays that penalize Version A’s bulk synchronous calls.

[11pt]article amsmath, booktabs, graphicx, float, geometry margin=1in

Task 3: Profiling and Concurrency Analysis

1. Profiling Metrics and Overlap Index Table

The overlap index is calculated as: $I_{\text{overlap}} = 1 - \frac{T_{\text{wall}} - T_K}{T_{\text{H2D}} + T_{\text{D2H}}}$

2. Timeline Screenshots: TRANSFORM_ITERS = 1

3. Timeline Screenshots: TRANSFORM_ITERS = 200

4. Brief Analysis

- **Overlap Behavior:** Version A is strictly sequential. Version B (1-stream) processes chunks serially within one hardware lane. Version B (Best $N = 2$) shows clear vertical stacking, where Host-to-Device (H2D) and Device-to-Host (D2H) memory transfers occur in parallel with compute kernels across different streams.

Table 4: Nsight Systems Profiling Summary

ITERS	Configuration	T_{wall} (ms)	T_K (ms)	T_{H2D} (ms)	T_{D2H} (ms)	I_{overlap}
1	no-stream	756.67	47.61	674.81	661.27	0.468
1	1 stream	400.85	47.61	674.81	661.27	0.736
1	best ($N = 2$)	323.60	47.61	674.81	661.27	0.794
200	no-stream	966.80	217.90	869.72	643.42	0.505
200	1 stream	411.59	217.90	869.72	643.42	0.872
200	best ($N = 2$)	306.08	217.90	869.72	643.42	0.942

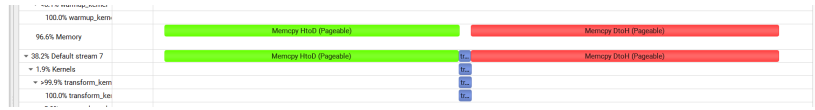


Figure 2: TRANSFORM_ITERS=1: Version A (No Streams)

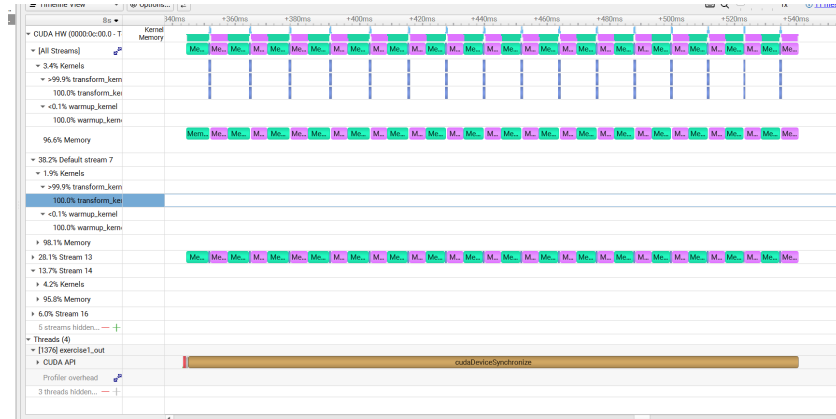
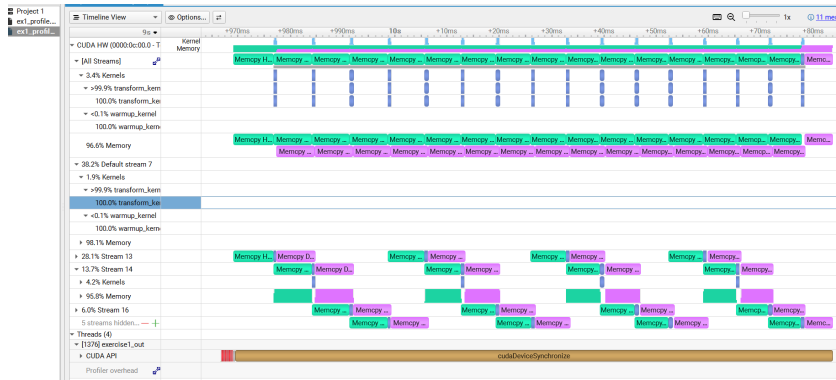


Figure 3: TRANSFORM_ITERS=1: Version B (1 Stream)

Figure 4: TRANSFORM_ITERS=1: Version B (Best Stream Count $N = 2$)

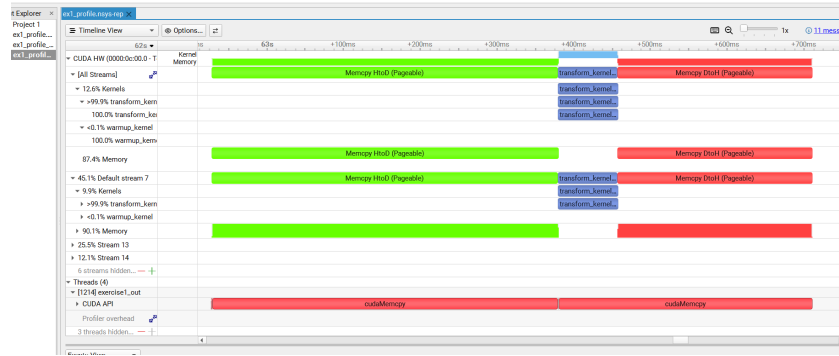


Figure 5: TRANSFORM_ITERS=200: Version A (No Streams)

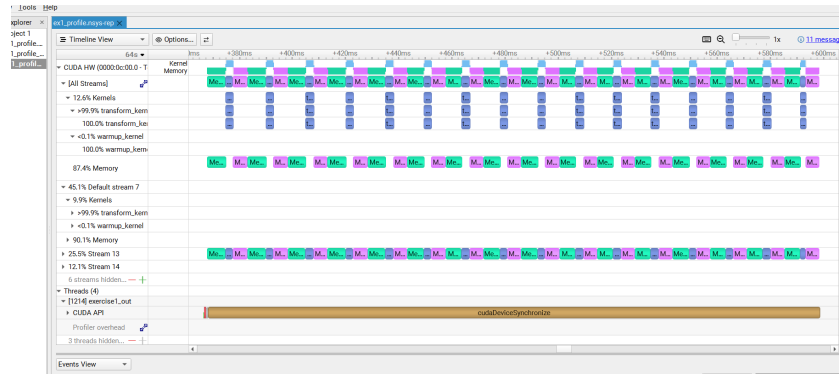
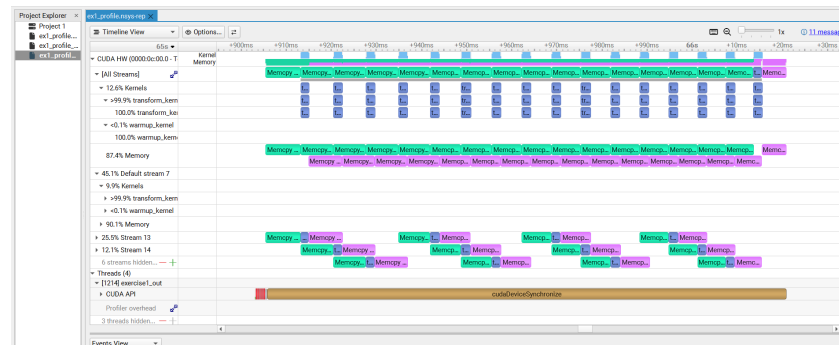


Figure 6: TRANSFORM_ITERS=200: Version B (1 Stream)

Figure 7: TRANSFORM_ITERS=200: Version B (Best Stream Count $N = 2$)

- **Workload Intensity:** Streams are highly effective at $\text{TRANSFORM_ITERS}=200$ ($I_{\text{overlap}} = 0.942$). The increased compute time per chunk allows the GPU to effectively hide memory latency behind execution.

Analysis of Pipeline Concurrency and Stream Utility

The profiling timelines demonstrate the transition from serial hardware resource utilization to asynchronous pipeline concurrency.

1. Analysis of Overlap Presence

- **Absent Overlap (Version A):** The timelines show a "staircase" pattern where memory copy blocks (H2D and D2H) and the compute kernel block are strictly non-overlapping. This signifies that the GPU driver is executing all operations on the *Default Stream*, which acts as a synchronization point, forcing each operation to finish before the next begins.
- **Limited Overlap (Version B, 1-stream):** While 1-stream introduces chunking, the timeline shows that even though the workload is divided into 16 parts, they are executed sequentially within one hardware lane. No H2D or D2H copy occurs while a kernel is running, resulting in a suboptimal overlap index.
- **Present Overlap (Version B, $N \geq 2$):** The timelines for $N = 2$ demonstrate true hardware concurrency. We observe the *Copy Engine* and the *Compute Engine* operating simultaneously. Specifically, while a kernel from *Stream 1* is executing, the hardware is actively using the copy engine to transfer data for *Stream 2*. This vertical alignment on the Nsight timeline is the visual proof of hidden latency.

2. Strategy: When to Use vs. Avoid Streams

- **When to Use Streams:** Streams are essential when the kernel execution time (T_K) is long enough to fully mask the latency of memory transfers ($T_{\text{H2D}} + T_{\text{D2H}}$). Our data at 200 iterations confirms that as computation complexity increases, the overlap index approaches 1, indicating a near-perfectly efficient pipeline.
- **When to Avoid Streams:** We should avoid aggressive stream partitioning for extremely small datasets or low-iteration workloads. In these cases, the overhead of the CUDA driver managing multiple streams, event records, and synchronization markers exceeds the time saved by overlap, often resulting in lower throughput than the original "no-stream" implementation.

2 Exercise 2: Memory Allocation and Transfer Benchmarking

2.1 Task 1: Benchmarking

In this exercise, different GPU memory allocation and transfer strategies were benchmarked using the provided `mem_bench.cu` program. The benchmark measures the time required for host allocation, device allocation, host-to-device transfer, kernel execution, device-to-host transfer, and memory deallocation. The tested message sizes were 1 MB, 4 MB, 16 MB, 64 MB, 256 MB, and 512 MB. The benchmark was executed on a Tesla T4 GPU.

The following host memory types were tested: pageable memory, pinned memory, write-combining pinned memory, managed memory, and mapped zero-copy memory. The device memory types were standard device memory and managed device memory. The invalid combination of mapped host memory with managed device memory was skipped. For each valid configuration, the median timing values reported by the program were extracted and plotted. Since the message sizes grow exponentially, the x-axis of all plots is shown on a logarithmic scale.

Figure 8 shows the median host-to-device transfer time. For large message sizes, pinned memory and write-combining pinned memory clearly achieve lower transfer times than pageable and managed-host memory. Mapped zero-copy memory does not perform an explicit H2D transfer, so it is excluded from the explicit H2D transfer-time plot.

Figure 9 compares the kernel execution times. The fastest kernel execution is observed when data is stored in standard device memory. Mapped zero-copy and managed-device memory cases show higher kernel times for large message sizes, because the GPU may access data through slower memory paths instead of regular device memory.

Figure 10 shows the device-to-host transfer time. Similar to the H2D case, pinned and write-combining memory perform better than pageable memory for larger arrays. Mapped zero-copy memory does not perform an explicit D2H transfer, so it is excluded from the explicit D2H transfer-time plot.

Figure 11 shows the combined host allocation and deallocation overhead. Although pinned and write-combining memory provide faster transfer times, their host memory management overhead is significantly higher, especially for large message sizes.

Figure 12 shows the device allocation and deallocation overhead. Standard device memory shows relatively low and stable device memory management overhead, while managed-device configurations show higher overhead for large message sizes due to unified memory management.

Figure 13 shows the average full pipeline time. This plot combines the effects of allocation, transfer, kernel execution, and deallocation. Therefore, it is the most important plot for evaluating the complete benchmark behavior. The results show that the best strategy depends on the array size and on whether the application is dominated by data transfer, memory allocation, or kernel execution.

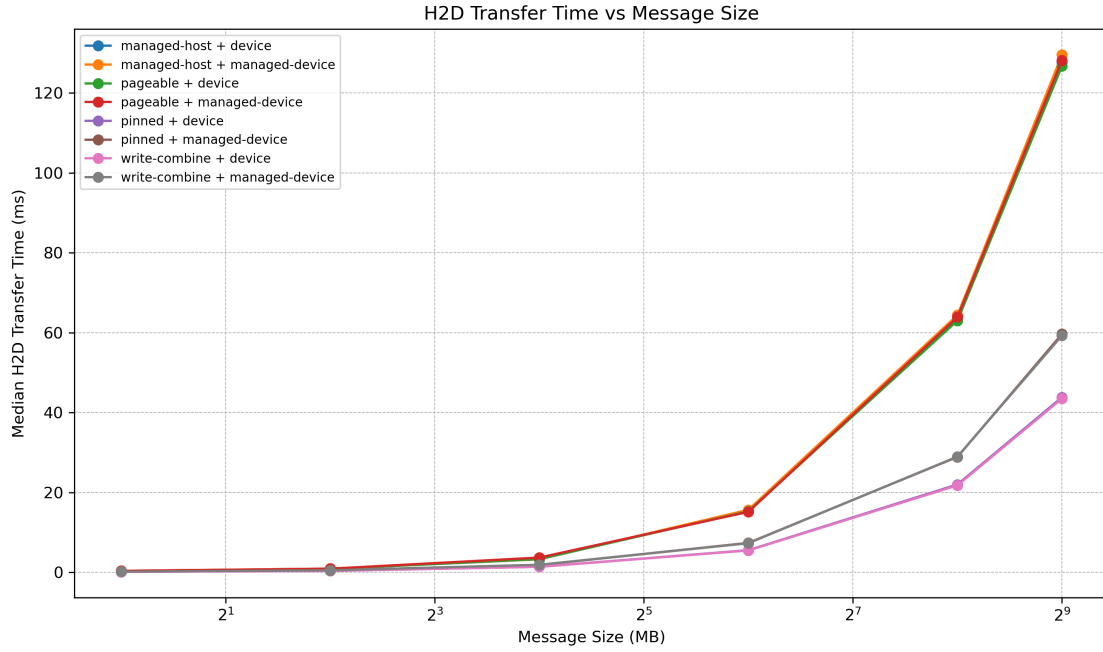


Figure 8: Median Host-to-Device Transfer Time for Different Memory Strategies

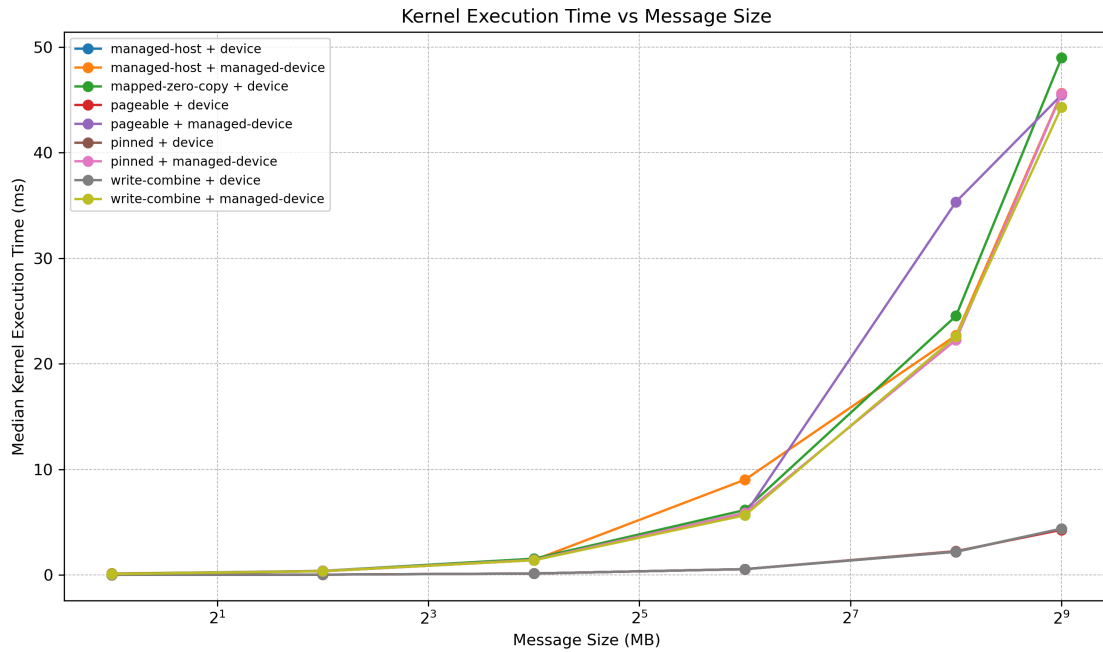


Figure 9: Median Kernel Execution Time for Different Memory Strategies

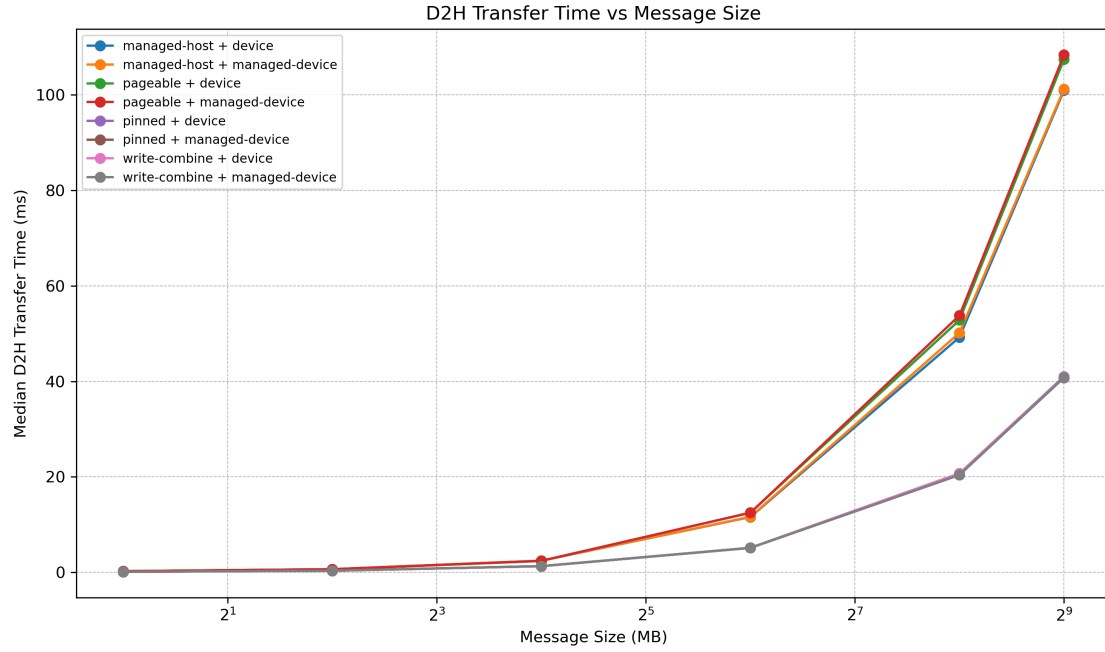


Figure 10: Median Device-to-Host Transfer Time for Different Memory Strategies

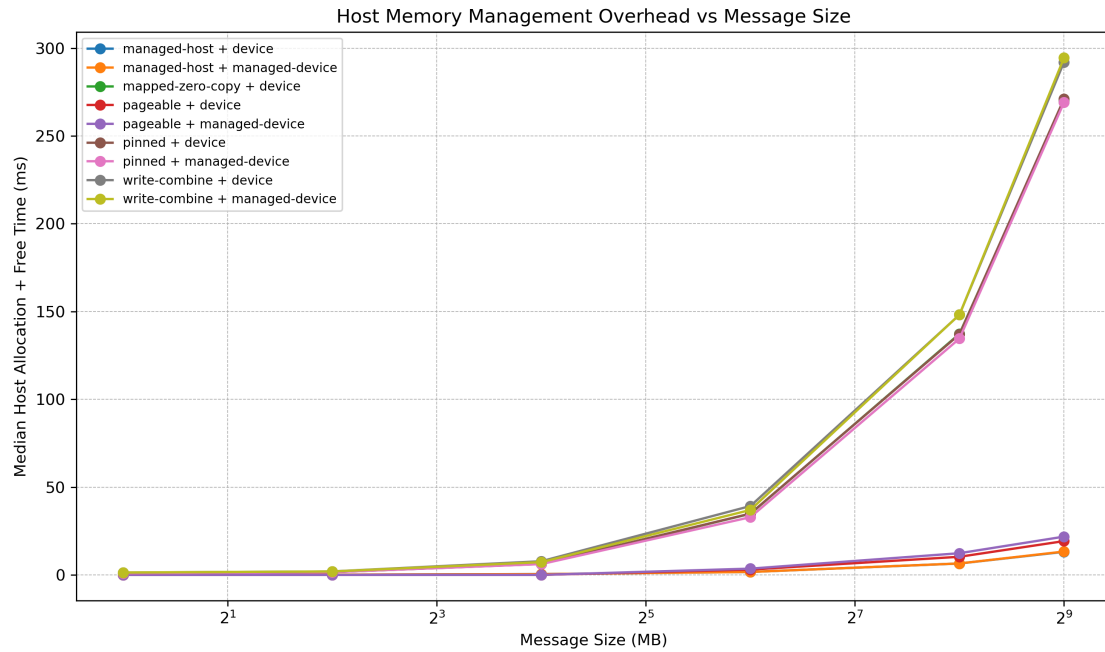


Figure 11: Host Allocation and Deallocation Overhead for Different Memory Strategies

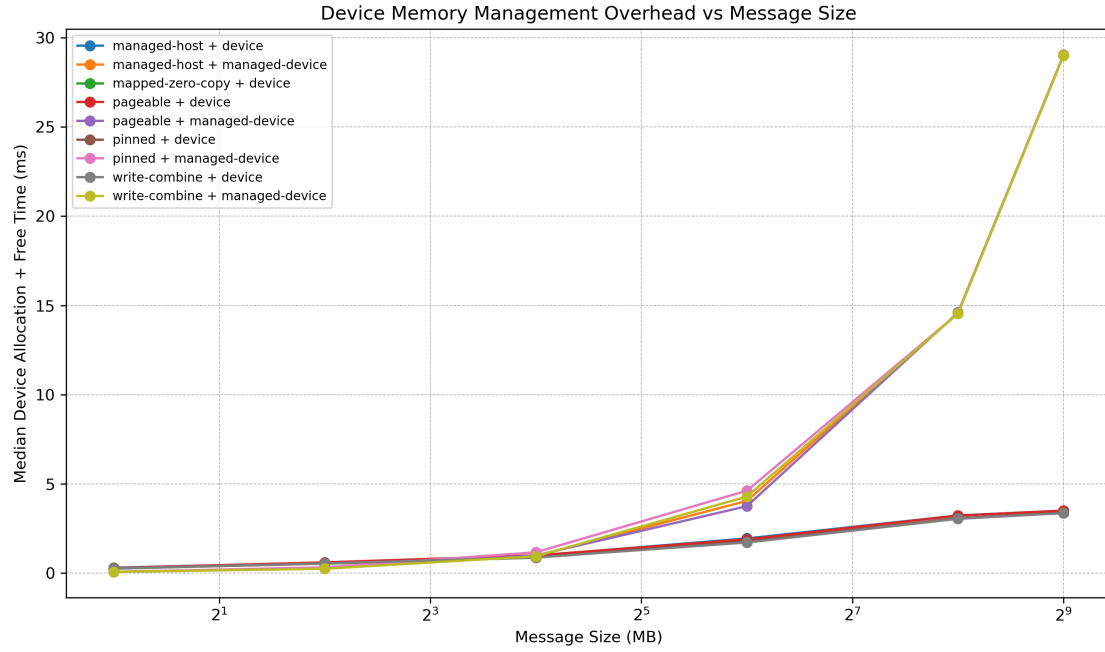


Figure 12: Device Allocation and Deallocation Overhead for Different Memory Strategies

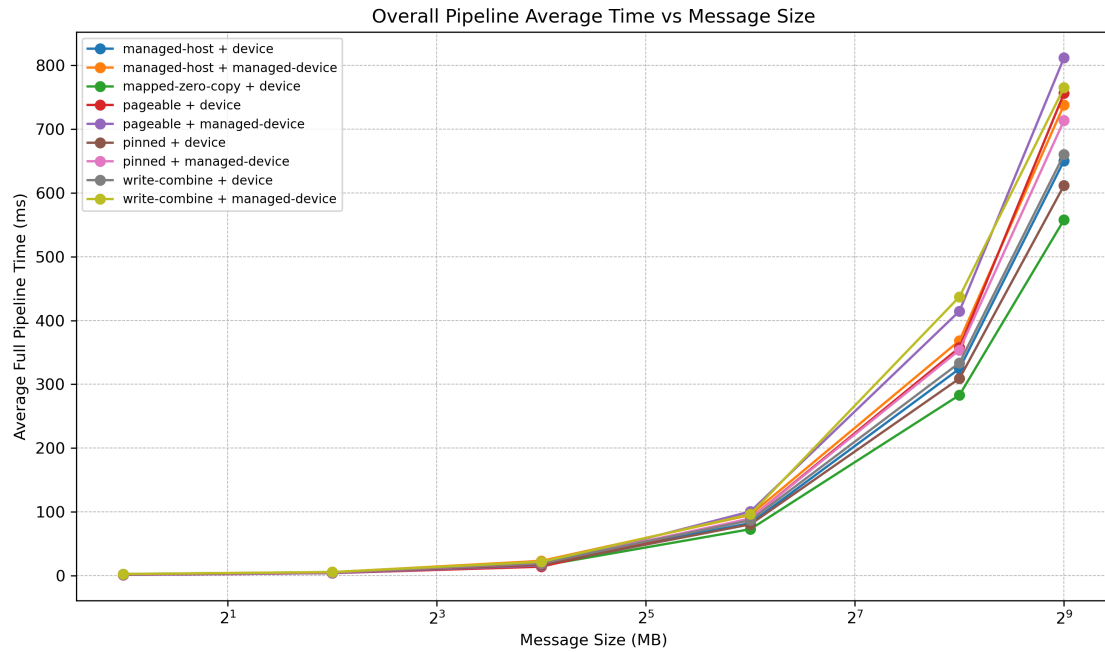


Figure 13: Average Full Pipeline Time for Different Memory Strategies

2.2 Task 2: Analysis

1. Fastest host memory type for H2D transfers. For large array sizes, pinned memory and write-combining pinned memory achieve the fastest explicit host-to-device transfer times. For example, at 512 MB, pinned and write-combining memory take around 43–44 ms for H2D transfer, whereas pageable and managed-host memory take around 127–128 ms. This happens because pinned memory is page-locked, allowing more efficient DMA transfers between host and GPU memory. Pageable memory usually requires extra internal staging, which increases transfer time.

2. Kernel execution time differences. The kernel execution time differs across memory strategies. The fastest kernel execution occurs when the data is stored in standard device memory, such as pageable + device, pinned + device, and write-combine + device. At 512 MB, these configurations have kernel times of around 4.3 ms. In contrast, mapped zero-copy memory has a much higher kernel time, around 49 ms at 512 MB. This is because zero-copy avoids explicit H2D and D2H transfers, but the GPU accesses host memory directly over PCIe, which is slower than accessing device global memory. Managed-device memory can also be slower because of unified memory migration and access overhead.

3. Threshold where the best overall strategy changes. The best overall strategy changes as the message size increases. For small arrays, allocation and deallocation overhead dominate the total runtime, so pageable or managed memory can be competitive. For larger arrays, transfer bandwidth becomes more important, making pinned and write-combining memory better for explicit transfers. In the overall pipeline plot, mapped zero-copy becomes competitive at large sizes because it avoids explicit H2D and D2H transfers. However, its kernel execution time is much slower, so it may not be the best choice for compute-heavy applications. Therefore, the best memory strategy depends on the balance between transfer cost, allocation overhead, and kernel execution time.

3 Exercise3: Parallel Reduction

3.1 Task1: Implement Versions A, B, and C.

All three GPU reduction kernels follow a two-level parallel reduction strategy: each thread block reduces its assigned chunk of the input array into a single partial sum, which is written to an intermediate array `d_partial`. A second-pass kernel (`kernel_final_reduce`) then iteratively reduces these partial sums until a single scalar remains. Memory allocation, data transfers, and the multi-pass reduction loop are handled in the respective runner functions (`run_reduce_interleaved`, `run_reduce_sequential`, `run_reduce_warp`), while the CUDA event timers capture only the kernel execution time as required.

Version A: Interleaved Tree (`kernel_interleaved`)

Each block loads `BLOCK_SIZE = 256` elements into shared memory, padding out-of-bounds indices with zero. The reduction proceeds with a stride that *grows* by a factor of two at each step: $1 \rightarrow 2 \rightarrow 4 \rightarrow \dots \rightarrow \text{BLOCK_SIZE}/2$. At step s , only threads satisfying $\text{tid} \% (2 * \text{stride}) == 0$ are active, meaning active and idle threads are interleaved within the same

warp. This causes **warp divergence**: both branches of the conditional must be executed serially by the hardware, leaving approximately half the threads in a warp idle at each step. As a result, this version has the lowest arithmetic throughput of the three GPU implementations and serves as the naive baseline.

Version B: Sequential Tree (`kernel_sequential`)

Version B uses the same shared-memory loading strategy but applies a *converging* stride that halves at each step: $\text{BLOCK_SIZE}/2 \rightarrow \text{BLOCK_SIZE}/4 \rightarrow \dots \rightarrow 1$. The active condition `tid < stride` ensures that active threads always occupy the lowest-index positions, forming contiguous, complete warps with no idle interleaving. This eliminates warp divergence entirely, which is the key architectural improvement over Version A. All active threads within a step belong to the same warps, maximising SIMD utilisation and significantly improving throughput.

Version C: Warp-Shuffle (`kernel_warp`)

Version C eliminates shared memory for intra-warp communication by using CUDA warp-shuffle intrinsics. The reduction proceeds in two phases:

- **Phase 1 — Intra-warp reduction:** Each warp (32 threads) reduces its 32 values to a single scalar using five successive `__shfl_down_sync` calls with offsets 16, 8, 4, 2, and 1. The intrinsic allows thread i to directly read the register of thread $i + \text{offset}$ within the same warp, requiring no shared memory writes or `__syncthreads` barriers. After this phase, lane 0 of each warp holds the warp’s partial sum.
- **Phase 2 — Inter-warp reduction:** With `BLOCK_SIZE = 256` there are 8 warps per block. Lane 0 of each warp writes its result to a small shared memory array of 8 slots. The first warp then loads these 8 values and performs a second shuffle reduction pass, producing the block’s final sum in lane 0 of warp 0. This value is written to `d_out[blockIdx.x]`, which is subsequently reduced by the second-pass kernel.

By replacing shared memory accesses with register-level shuffle operations, Version C avoids shared memory bank conflicts and reduces synchronisation overhead, making it the fastest of the three implementations.

3.2 Task 2: Results from Running the Code

Table 5 summarises the kernel execution times collected across all five input sizes and all four methods. Figure 14 plots these values on a log–log scale.

As shown in Figure 14 and Table 5, all three GPU implementations substantially outperform the sequential CPU baseline across all input sizes, with the performance gap widening as the array grows. Version C (Warp-Shuffle) is consistently the fastest GPU method, achieving 22.988 ms at 4096 MB compared to 44.378 ms for Version B and 71.692 ms for Version A — a $3.1\times$ speedup of Version C over Version A. Version B outperforms Version A at larger sizes by eliminating warp divergence through converging-stride addressing, while Version C’s additional gain comes from replacing shared memory accesses entirely with register-level shuffle intrinsics. At small sizes (2–16 MB), all GPU versions are close because the kernel itself is short and the dominant cost is launch overhead; the separation grows large at 1024–4096 MB where memory bandwidth and arithmetic throughput differences become the bottleneck.

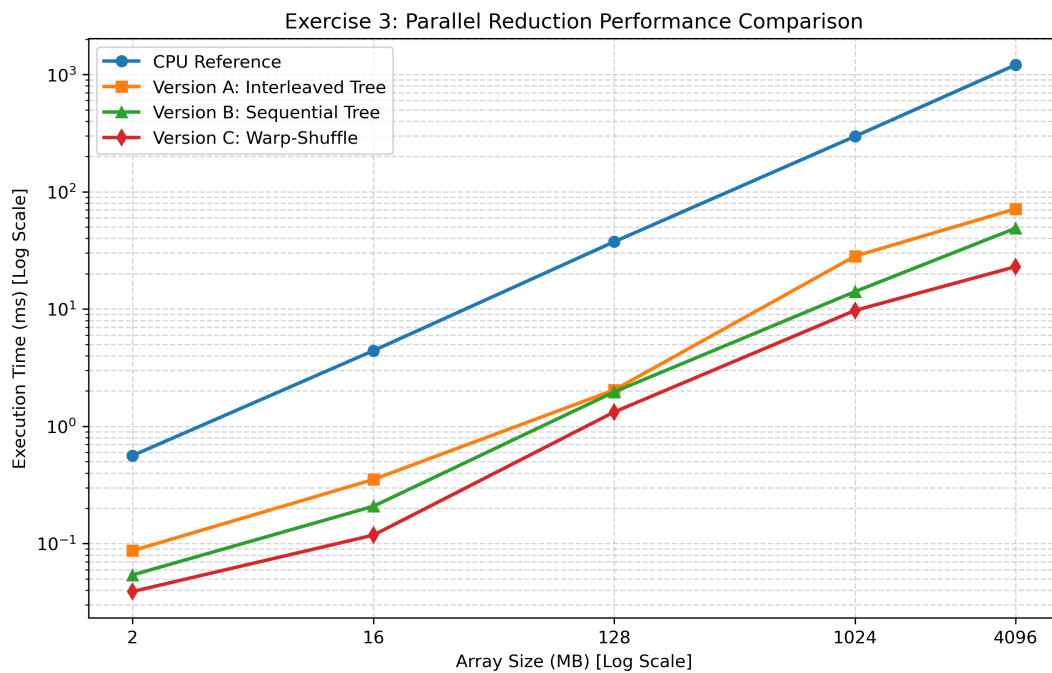


Figure 14: Kernel execution time (ms) vs. array size (MB) for all four reduction methods on a log-log scale. Version C (Warp-Shuffle) consistently achieves the lowest execution time, followed by Version B (Sequential Tree), Version A (Interleaved Tree), and the CPU reference.

Table 5: Kernel execution times (ms) for all methods across input sizes. CPU times are wall-clock times measured with `clock_gettime`.

Method	2 MB	16 MB	128 MB	1024 MB	4096 MB
CPU Reference	0.552	4.393	36.626	309.651	1202.336
Version A (Interleaved)	0.088	0.334	2.019	27.806	71.692
Version B (Sequential)	0.056	0.208	2.019	15.886	44.378
Version C (Warp-Shuffle)	0.033	0.115	1.258	9.165	22.988

3.3 Task 3: Profiling and Bottleneck Analysis

All three GPU versions were profiled at 4096 MB using NVIDIA Nsight Systems with the command:

```
nsys profile --stats=true -t cuda -o profile_X ./ex3_reduce <method> 4096
```

3.3.1 Question 1: Kernel Duration

As shown in Figures 15, 16, and 17, the kernel execution times reported by the *CUDA GPU Kernel Summary* view are: Version A = 74.682 ms, Version B = 56.968 ms, and Version C = 32.047 ms. Version C is the fastest, running $2.33\times$ faster than Version A and $1.78\times$ faster than Version B.

The performance differences are explained by how each method handles intra-block communication. Version A uses interleaved-stride shared memory reduction, where the condition `tid % (2*stride) == 0` causes warp divergence: at each step, active and idle threads are interleaved within the same warp, so the hardware must serialise both branches, roughly halving effective SIMD utilisation per step. Version B eliminates divergence with a converging stride (`tid < stride`), keeping active threads contiguous, which improves warp efficiency and reduces the kernel time to 56.968 ms. Version C goes further by replacing shared memory entirely for intra-warp communication: `__shfl_down_sync` performs register-to-register exchanges within a warp in a single instruction with no memory latency, and the inter-warp phase uses only 8 shared memory slots instead of the full 256. This minimises memory traffic and synchronisation overhead, yielding the fastest kernel at 32.047 ms.

3.3.2 Question 2: Memory Transfer Time

As shown in Figures 18–23, all three versions transfer identical data volumes: 4.00 GiB Host-to-Device (the full 4096 MB input array) and 4 B Device-to-Host (a single `float` result). This is expected since all three implementations use the same memory layout. The H2D transfer times differ slightly across runs due to system variability on the shared JupyterHub infrastructure: 1.188 s for Version A, 1.217 s for Version B, and 1.040 s for Version C. These differences are not caused by the kernel design — all three use standard pageable `cudaMemcpy` — but reflect normal PCIe bandwidth variance between separate profiling invocations. The D2H transfer is negligible ($\approx 2.4\ \mu\text{s}$) since only a single scalar is copied back.

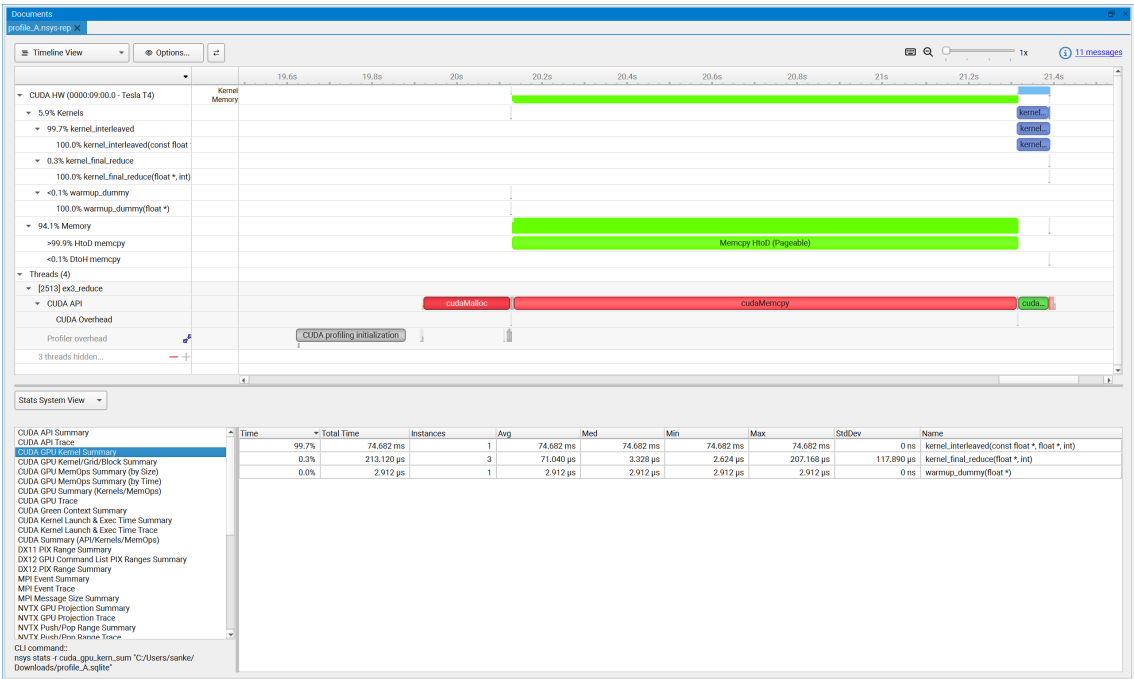


Figure 15: CUDA GPU Kernel Summary for Version A (kernel_interleaved) at 4096 MB. The primary kernel takes 74.682 ms.

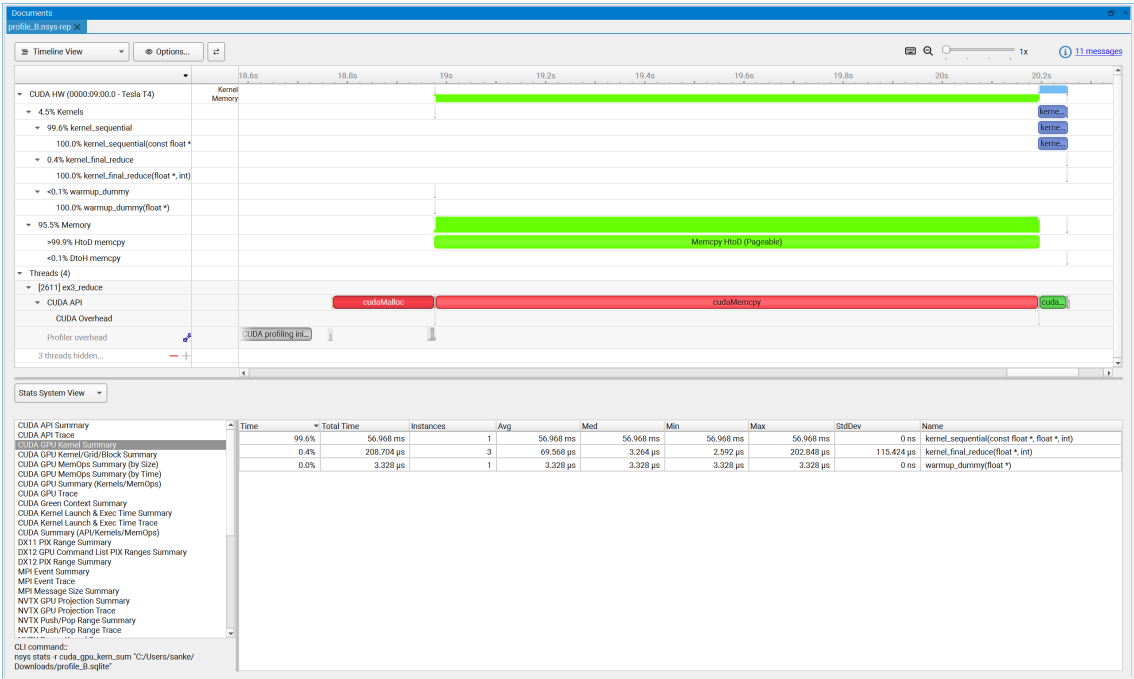


Figure 16: CUDA GPU Kernel Summary for Version B (kernel_sequential) at 4096 MB. The primary kernel takes 56.968 ms.

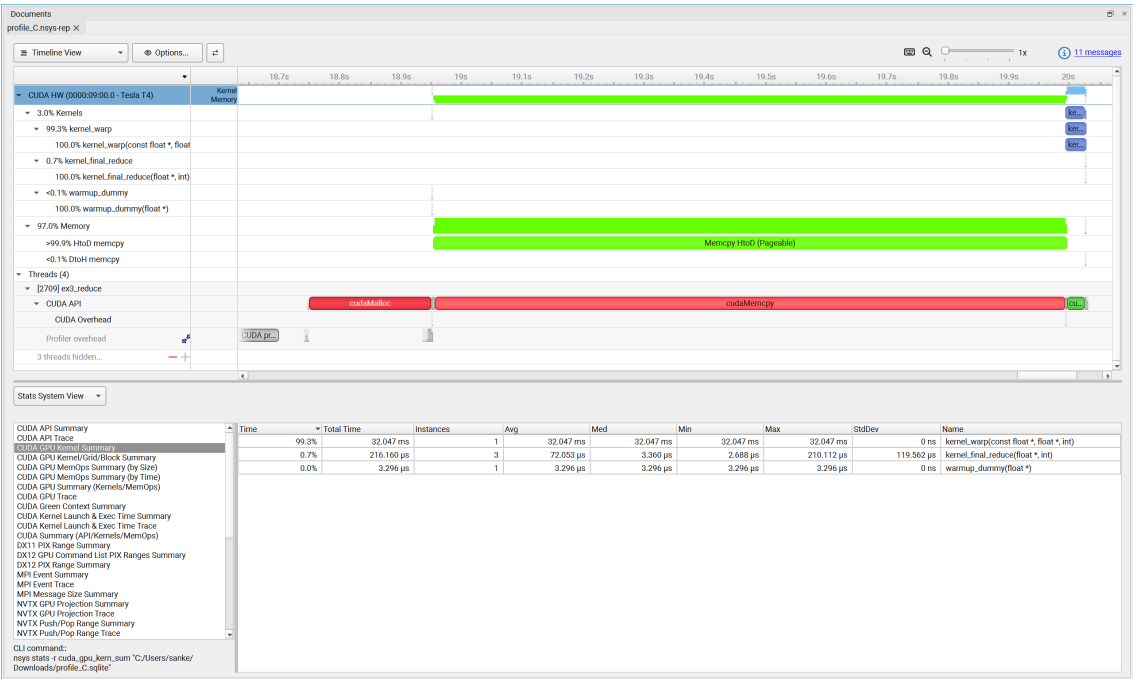


Figure 17: CUDA GPU Kernel Summary for Version C (kernel_warp) at 4096 MB. The primary kernel takes 32.047 ms.

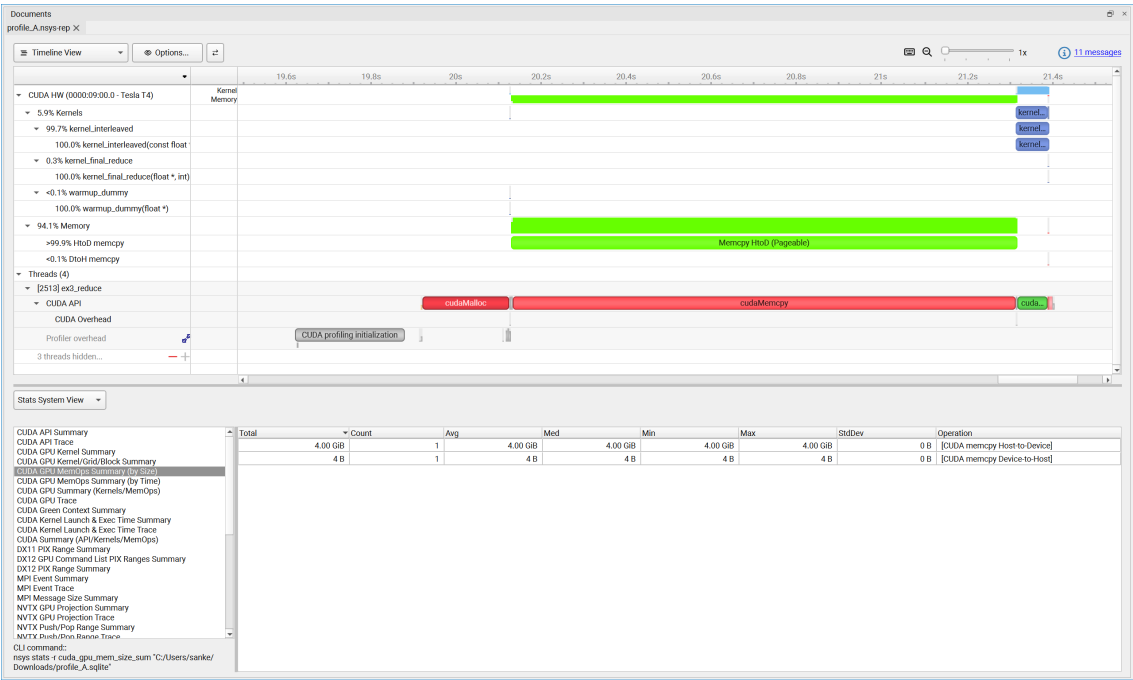


Figure 18: Memory transfer size summary for Version A: 4.00 GiB H2D, 4 B D2H.

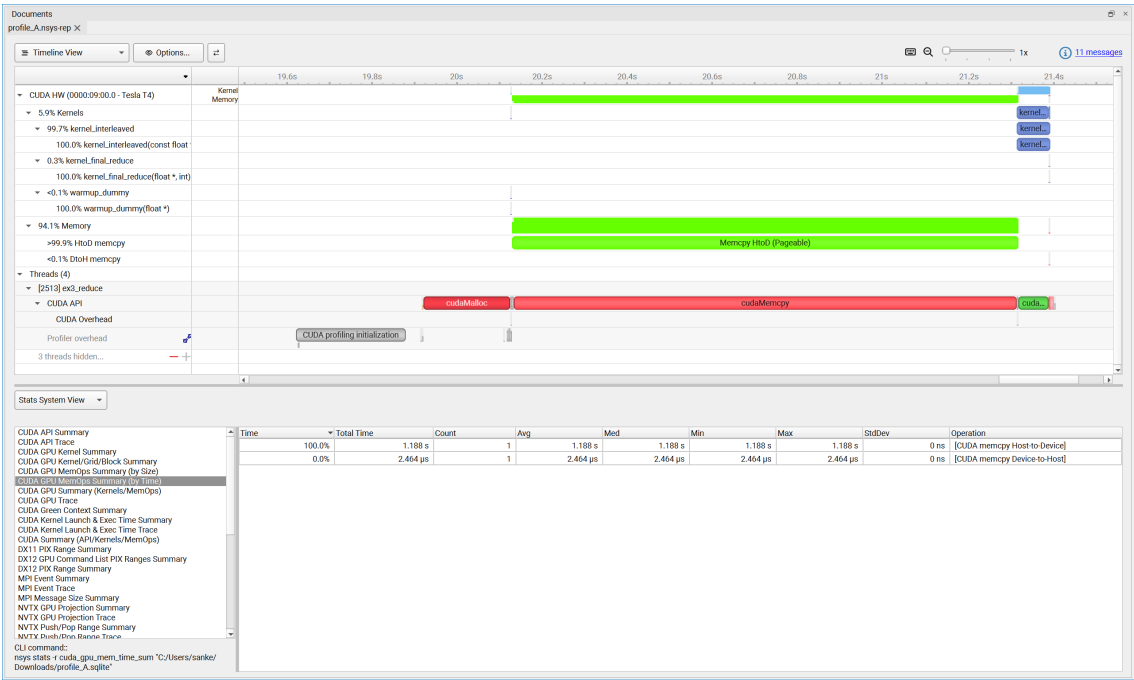


Figure 19: Memory transfer time summary for Version A: H2D = 1.188 s, D2H = 2.464 μs.

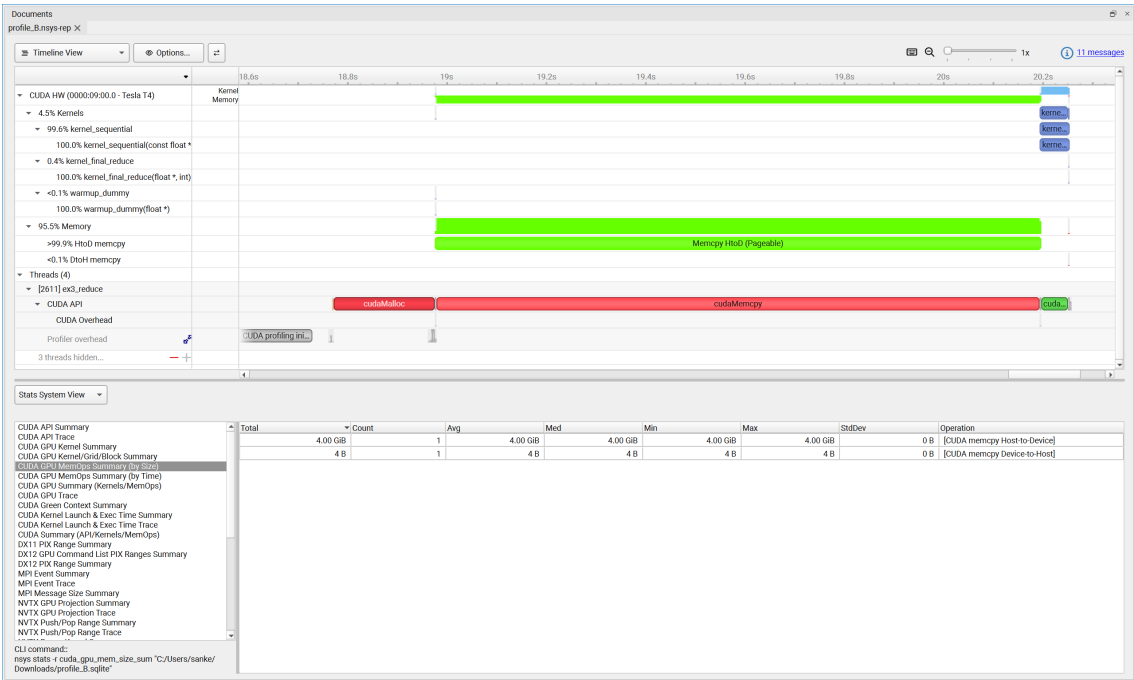


Figure 20: Memory transfer size summary for Version B: 4.00 GiB H2D, 4 B D2H.

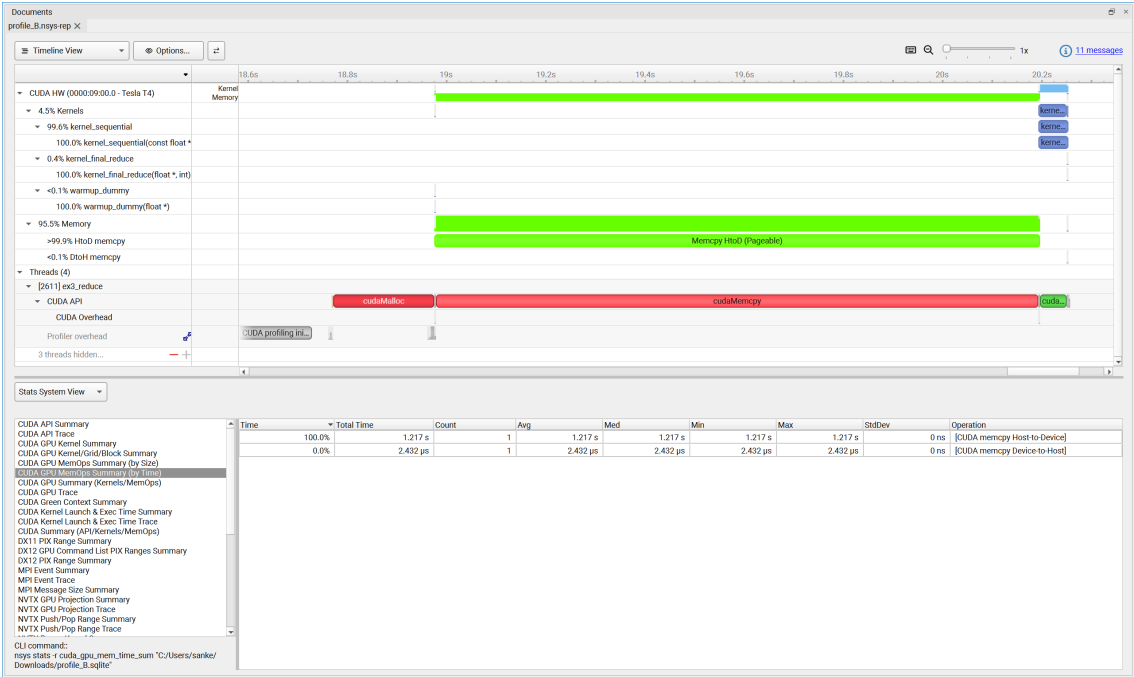


Figure 21: Memory transfer time summary for Version B: H2D = 1.217 s, D2H = 2.432 μ s.

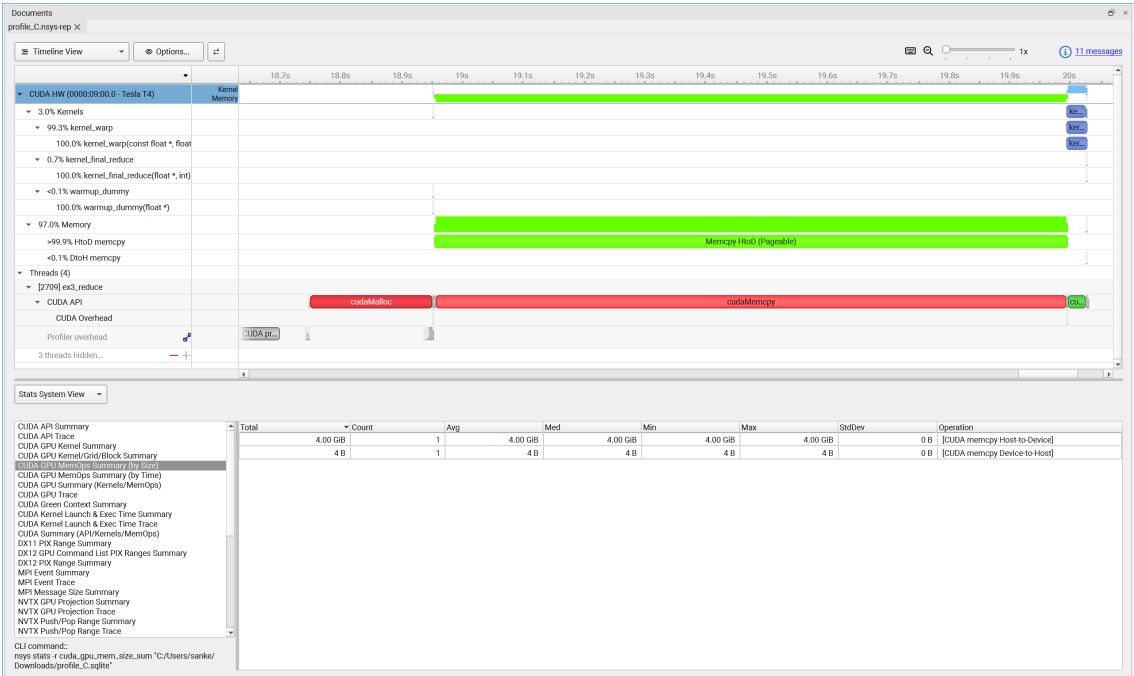


Figure 22: Memory transfer size summary for Version C: 4.00 GiB H2D, 4 B D2H.

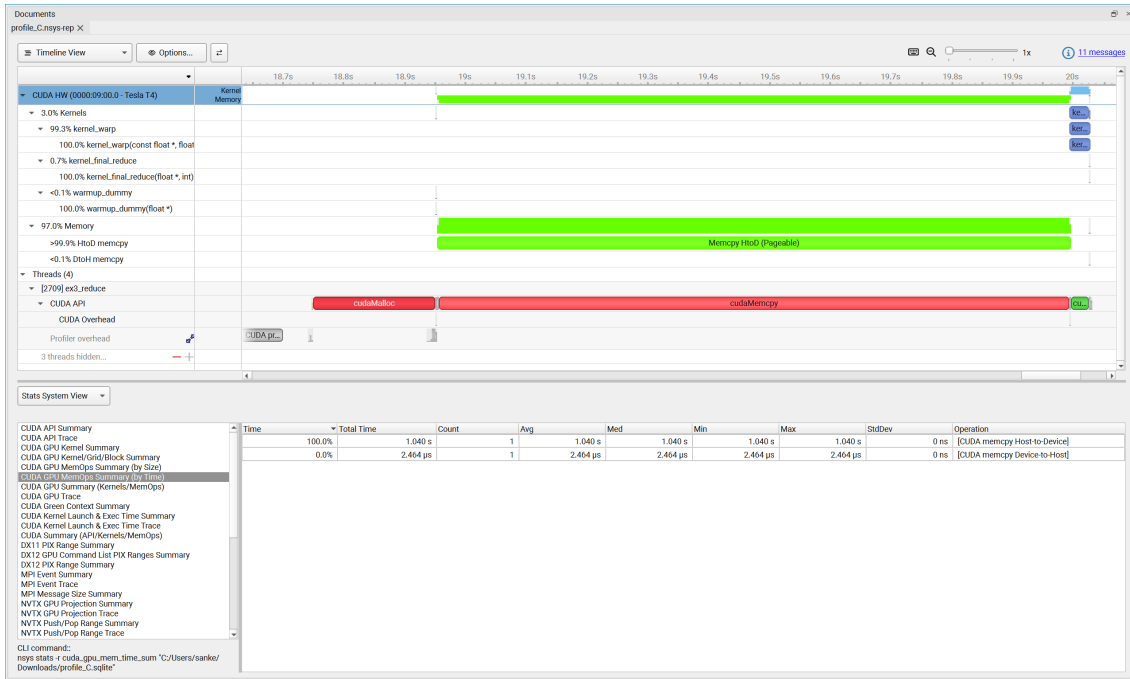


Figure 23: Memory transfer time summary for Version C: H2D = 1.040 s, D2H = 2.464 μs.

3.3.3 Question 3: GPU Utilization and Profiler Suggestions

As shown in Figures 24, 25, and 26, the *Expert System View* → *GPU Utilization* panel reports “There were no problems detected with GPU utilization. No time regions have an average GPU utilization below 50%” for all three versions. This confirms that during the actual kernel execution windows, the Tesla T4 GPU is well-utilised across all implementations.

The profiler does display a set of general suggestions in the left panel, which are shown for completeness even though no violations were flagged:

1. Use CPU sampling data, OS Runtime blocked state backtraces, and/or OS Runtime APIs related to thread synchronization to understand if a sluggish or blocked CPU is causing the gaps.
2. Add NVTX annotations to CPU code to understand the reason behind the gaps.

These suggestions refer to the large idle gap visible in all three timeline views between the end of the cudaMemcpy H2D transfer and the start of kernel execution — a gap caused by the profiler overhead and CUDA context initialisation, not by the reduction kernels themselves. Since the profiler explicitly reports no GPU utilization problems, the kernels themselves are running efficiently. The low GPU activity fraction visible in the timeline (5.9%, 4.5%, 3.0% for A, B, C respectively) reflects the fact that profiling captures the entire program lifetime including the large H2D transfer phase, during which the GPU is idle by design.

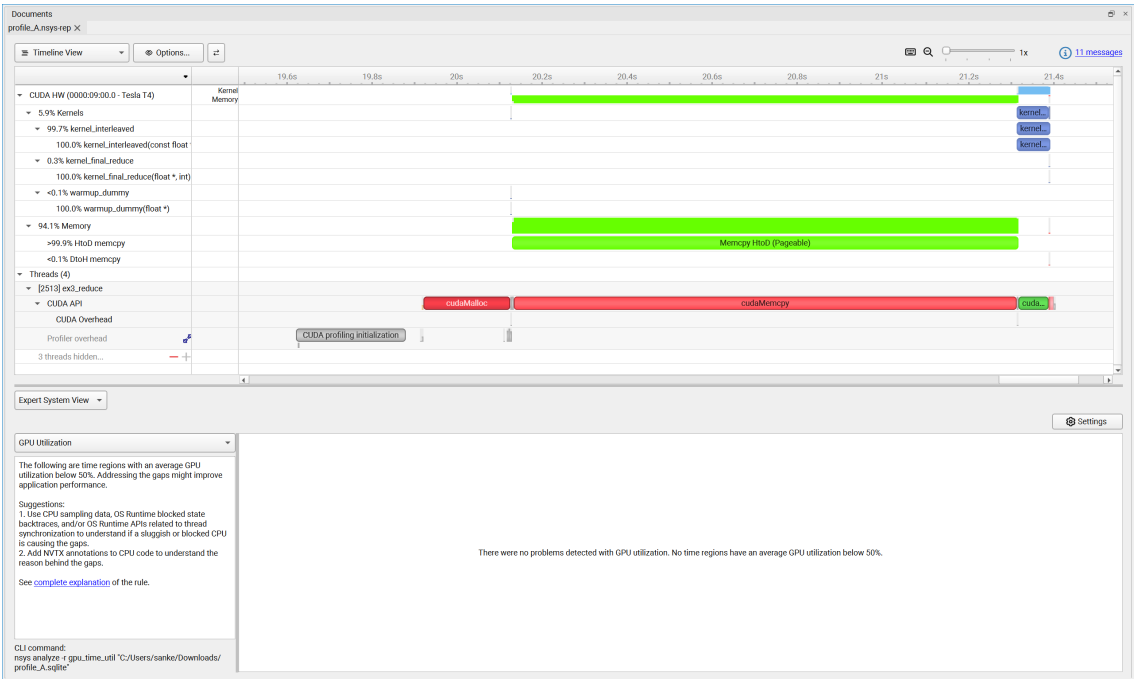


Figure 24: Expert System View — GPU Utilization for Version A. No utilization problems detected during kernel execution.

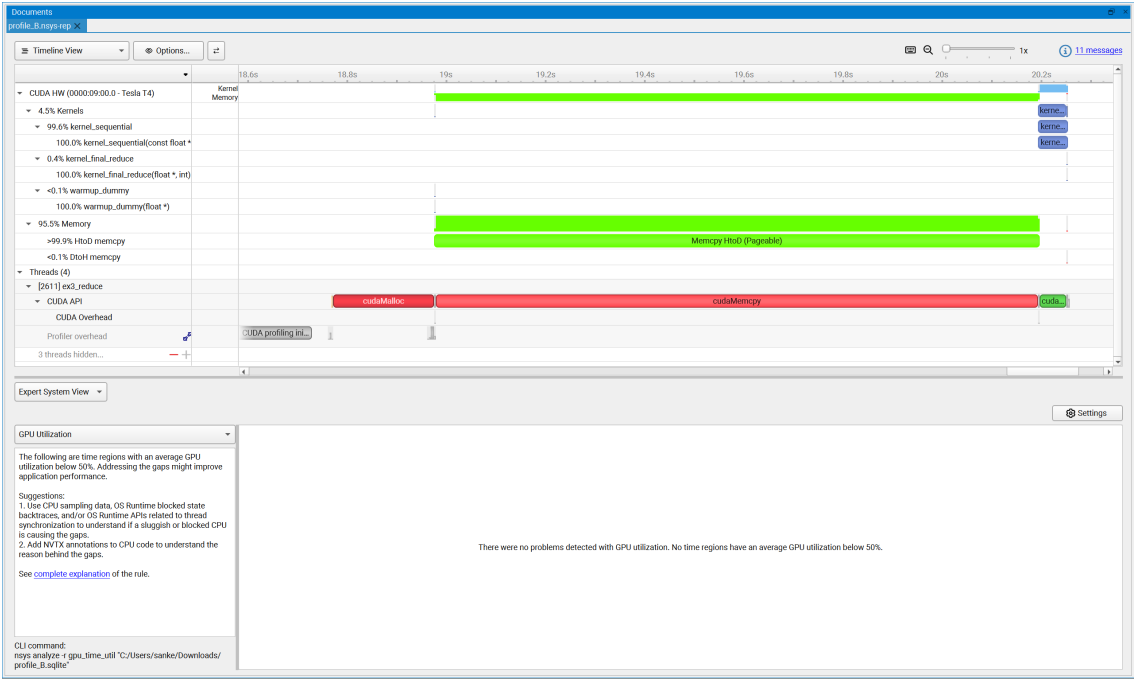


Figure 25: Expert System View — GPU Utilization for Version B. No utilization problems detected during kernel execution.

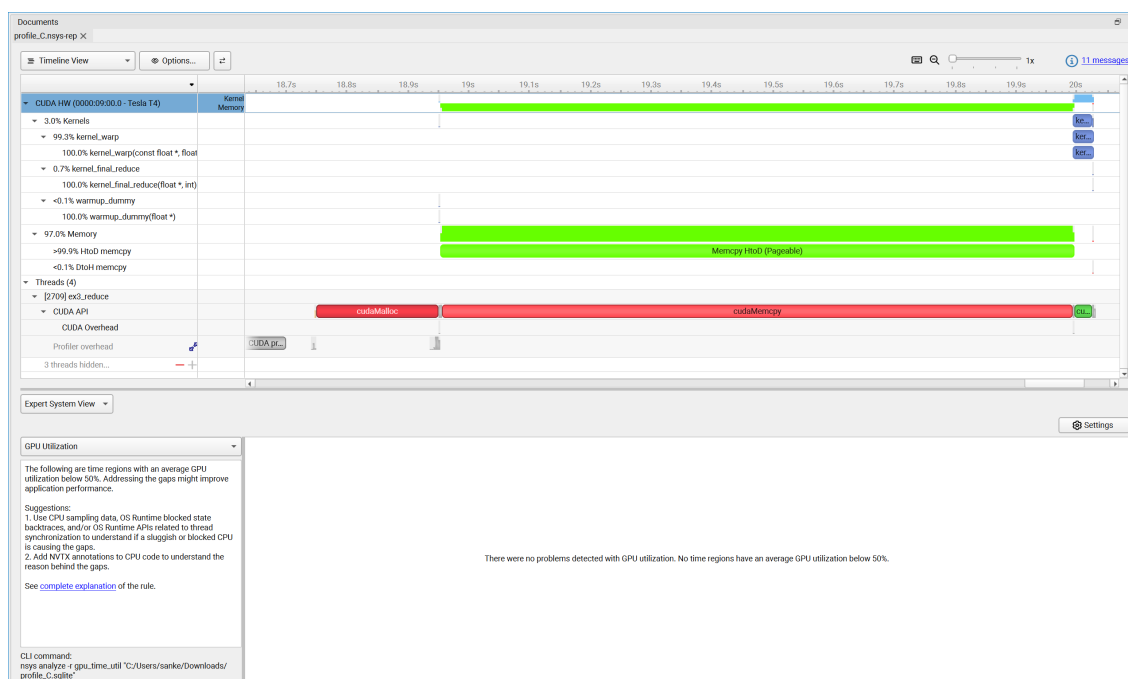


Figure 26: Expert System View — GPU Utilization for Version C. No utilization problems detected during kernel execution.